

Name : Bisma Fatima

Class : CS-A -24

Roll No : F24605036

Course : Data Structure

Code : CS- 210

Lecturer : Sir Mehran Yousaf

Assignment 01

Name: Bisma Fatima

Question No 01

Implement a Min stack

Program

```
#include <iostream>
```

```
using namespace std;
```

```
class Stack { // Class
```

```
private:
```

```
    int arr[100];
```

```
    int top;
```

```
    int size;
```

```
public:
```

```
Stack(int s=100) // constructor
```

```
{ size = s;
```

```
  top = -1;
```

```
}
```

```
bool isEmpty() // isEmpty()
```

```
{ return top == -1;
```

```
}
```

```
bool isFull() // isFull()
```

```
{ return top == size - 1;
```

```
}
```

```
void push(int value)           // push()
{
    if(isfull())
    {
        cout<<"Stack Overflow!";
        return;
    }
    arr[++top] = value;
}
```

```
void pop()                     // pop()
{
    if(isempty())
    {
        cout<<"Stack Underflow!";
        return;
    }
    top--;
}
```

```
int peek() {                   // peek()
    if(isempty())
    {
        cout<<"Stack is empty!";
        return -1;
    }
    return arr[top];
}
```

```
};
```

Name: Bisma

```
Class Minstack {           // Minstack class
    private:
        Stack mainstack;
        Stack minstack;
    public:
        void push (int x)    // push (int x)
        {
            mainStack.push(x);

            if (minstack.isEmpty() || x <= minstack.peek())
            {
                minstack.push(x);
            }
        }

        void pop()           // pop()
        {
            if (mainstack.isEmpty())
            {
                cout << "Stack is empty!";
                return;
            }

            int topval = mainStack.peek();
            mainStack.pop();
            if (topval == minstack.peek())
            {

```

```

        minstack.pop();
    }
}

int top()           // top()
{
    return minstack.peek();
}

int getMin()       // get Min()
{
    if (minstack.isEmpty())
    {
        cout << "In Stack is empty!";
        return -1;
    }
    return minstack.peek();
}

};

int main()         // Main class
{
    Minstack obj;

    obj.push(5);
    obj.push(2);
    obj.push(9);
    obj.push(1);

    cout << "Current Minimum: " << obj.getMin();

    obj.pop();
}

```


Name: Bisma

```
cout << "Minimum after pop:" << obj.getMin();  
    obj.pop();  
    cout << "Top element:" << obj.top();  
    cout << "Current Minimum:" << obj.getMin();  
  
    return 0;  
}
```

Dry Run

Initial state :

- mainstack is empty: []
- minstack is empty: []

Push Operations

1. obj.push(5)

- minstack : [5] [since minstack is empty, 5 is pushed]
- mainstack : [5]

2. obj.push(2)

- mainstack : [5, 2]
- minstack : [5, 2] ; As $2 < 5$ so push 2 to minstack.

3. obj.push(8)

- mainstack : [5, 2, 8]
- minstack : [5, 2] ; As $8 < 2$ false.

4. obj.push(1)

- mainstack : [5, 2, 8, 1]
- minstack : [5, 2, 1] ; As $1 < 2$ True so push.

Get Minimum -
obj.getmin() : return 1 [top element of minstack]

Pop operations:

1. obj.pop()

• main stack : [5, 2, 8] popped 1

• min stack : [5, 2] popped element 1.

• obj.getmin() : return 2 [new top of minstack]

2. obj.pop()

• main stack : [5, 2] popped 8

• min stack : [5, 2] No change, 8 wasn't in minstack.

• obj.top() : return 2.

• obj.getmin() : still 2.

Output

Current Minimum : 1

Minimum after pop : 2

Top element : 2

Current minimum : 2
